

Floating-Point Data Transformation for Lossless Compression

Samirasadat Jamalidinan
McMaster University
Hamilton, Canada
jamalids@mcmaster.ca

Kazem Cheshmi
McMaster University
Hamilton, Canada
cheshmi@mcmaster.ca

ABSTRACT

Floating-point data is widely used across various domains. Depending on the required precision, each floating-point value can occupy several bytes. Lossless storage of this information is crucial due to its critical accuracy, as seen in applications such as medical imaging and language model weights. In these cases, data size is often significant, making lossless compression essential. Previous approaches either treat this data as raw byte streams for compression or fail to leverage all patterns within the dataset. However, because multiple bytes represent a single value and due to inherent patterns in floating-point representations, some of these bytes are correlated. To leverage this property, we propose a novel data transformation method called Typed Data Transformation (TDT) that groups related bytes together to improve compression. We implemented and tested our approach on various datasets across both CPU and GPU. TDT achieves a geometric mean compression ratio improvement of $1.16\times$ over state-of-the-art compression tools such as zstd, while also improving both compression and decompression throughput by $1.18\text{--}3.79\times$.

1 INTRODUCTION

Floating-point data is prevalent in numerous domains, including scientific computing [19, 36] and machine learning [38]. Compressing this data offers significant advantages, primarily by reducing storage space and memory footprint. In many applications involving floating-point values, lossless compression is crucial. This is particularly true in sensitive areas such as medical imaging, where even minor data alterations can have significant diagnostic implications, and in machine learning models, where preserving the precision of weights and biases is essential for model accuracy. This paper focuses on the lossless compression of floating-point datasets.

General-purpose lossless compression methods such as snappy [15], zstd [13], and lz4 [8] treat all data, including floating-point values, as raw byte sequences for compression. However, typed data, such as floating-point numbers, is composed of multiple bytes that are often correlated, as they collectively represent a single value adhering to a specific standard. For instance, the IEEE 754 standard [17] is commonly used for floating-point representation, defining various type widths and allocating bytes for the exponent and mantissa. These components are themselves represented as integer or floating-point values. These standards inherently enforce partial patterns across floating-point values. However, these patterns are not explicitly exploited by generic compression tools because the data is treated simply as a stream of bytes.

Floating-point-based compression tools often capitalize on the unique properties of floating-point types for enhanced compression. These methods typically predict the next value based on prior values [21, 28] or a learned model [7, 23]. They then store only

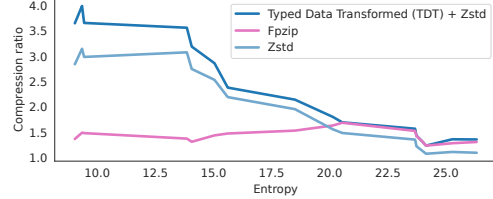


Figure 1: Typical compression behavior of floating-based compression tools such as fpzip and generic compression tools such as Zstd and how TDT provides best of both worlds when combined with generic compression tools.

the differing bits, often using an XOR operation, which leverages the smooth transitions commonly observed in floating-point data streams. For instance, Gorilla [28] and Chimp [21] exploit the frequent similarity of leading and trailing bits in consecutive floating-point values, making them highly compressible. While effective at saving space, these approaches often overlook the global patterns across bytes that generic compression methods leverage. Figure 1 illustrates this by comparing fpzip [23], a floating-point-specific compression tool, with zstd [13], a generic compression tool, across real-world floating-point datasets. As the figure shows, zstd effectively exploits the low-entropy properties of these datasets, while fpzip performs better on high-entropy datasets. This comparison highlights a performance gap between these two classes of methods.

Data transformation is a known technique for improving compression ratios. A data transform rearranges a dataset to expose more frequent patterns, thereby facilitating the application of other compression methods. While a data transformation does not alter the fundamental entropy of the data, it makes patterns easier to identify, which significantly aids compression methods such as dictionary coding. The Burrows-Wheeler Transform (BWT) [32], for example, is commonly applied to string datasets, such as genomic data, to generate more repetitive patterns. However, data transformation for floating-point datasets remains less explored. Shuffle and Bitshuffle [26] are notable examples of data transforms specifically applied to floating-point datasets. These methods reorder data before a subsequent compression technique is applied. Their key limitation, however, is their uniform treatment of all data, reordering bits and bytes identically across different applications, even though the relationship between bits and bytes can vary depending on data types and specific application contexts.

This paper proposes Typed Data Transformation (TDT), a novel data transformation technique designed to enhance the compression of floating-point data by enabling generic compression tools to leverage floating-point characteristics. TDT employs a new clustering technique to group correlated bytes within a dataset and

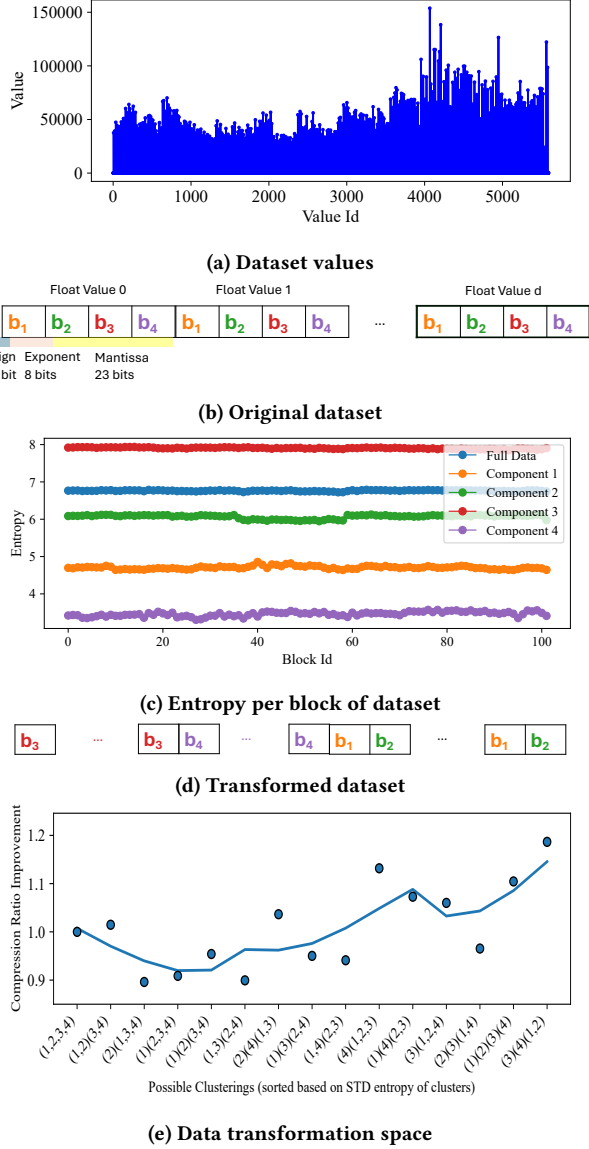


Figure 2: a) Range of values in a floating-point dataset. b) The byte representation of the dataset in memory. Four bytes store a float value, i.e., single precision, using the IEEE 754 standard. c) The entropy of the dataset and each byte across the dataset, i.e., components. d) The byte representation of the dataset after applying TDT transformation. e) All possible byte clustering for the dataset and their compression ratios.

then reorders these bytes to ensure that related information is stored contiguously. Also, TDT facilitates independent compression and decompression of bytes, which improves load balance and, hence, throughput. We conducted an extensive evaluation of this method across datasets from High-Performance Computing (HPC), Observation (OBS), database transactions (DB), and time-series (TS) domains. The methodology was also tested with a wide range of



Figure 3: Overall view of decomposed typed data transformation (TDT) and its application to de/compression

generic compression techniques, including lz4, zstd, zlib, snappy, bzip, and nvCOMP. Across all tested datasets, TDT achieved a geometric mean (GMean) compression ratio of 1.86×, outperforming the GMean of 1.62× achieved by the baseline compression methods. Additionally, TDT inherently creates more parallelizable workloads, thereby amortizing the cost of transformation on parallel resources and significantly improving overall compression and decompression throughput.

2 MOTIVATION AND OVERVIEW

This section motivates how TDT groups bytes using an example and provides an overview of the designed framework. A part of the *solar_wind* dataset from TS is shown in Figure 2a. Each sample is shown with a single-precision floating-point type represented using the IEEE 754 standard. Figure 2b also shows how the float values are laid out in memory using the IEEE standard. The single-precision (float32) format uses 32 bits divided into three fields: a 1-bit sign (s), an 8-bit exponent (e), and a 23-bit mantissa (m). In this format, each floating-point value v is represented as $v = (-1)^s \times 2^{e-bias} \times (1 + m/2^{23})$, where $bias$ is 127.

The entropy of each byte often differs in floating-point values due to their underlying storage standard and common data patterns found in scientific and machine learning datasets. A higher entropy value indicates a more random data distribution and, thus, lower compressibility. Figure 2c illustrates the entropy of each byte (component) within the *solar_wind* dataset, alongside the dataset's overall entropy. As the figure shows, the entropy of individual bytes varies significantly from the dataset's total entropy. Specifically, components 1, 2, and 4 exhibit lower entropy than the dataset's overall entropy, while the third byte has higher entropy. The dataset's overall entropy is calculated assuming each symbol is one byte long. This non-uniform distribution of entropy can be used to reorganize the data, reducing the search space for compression, rather than simply looking for similar entropy components across the dataset.

Figure 2d illustrates how the components are rearranged based on clustering entropy. Bytes 1 and 2 have more similarity; placing them contiguously creates data with more uniform entropy. Bytes 3 and 4 are distinct and should be separated. After the reordered data is created, bytes (components) with close entropies are formed to improve compressibility by a compression method. For example, using zstd, as an efficient compression method, to compress the original dataset and the transformed dataset leads to a compression ratio of 1.44× and 1.71×, respectively.

Deciding how to cluster bytes is challenging as the number of clusters is not known, and finding interaction between bytes are data dependent. Figure 2e shows all possible clustering for four bytes and their associated compression ratios. The x-axis is sorted based standard deviation (STD) of entropies in components to show how non-uniformity correlates with compression ratios. To address the discussed challenges in data transformation, this paper

Algorithm 1: TDT Algorithm

```

Input :  $D$ 
Output:  $P$ 
/* Step I: Feature Extraction (Section 3.2) */
1  $X \leftarrow \text{featureExtraction}(D)$ 
/* Step II: Clustering (Section 3.3) */
2  $L \leftarrow \text{Linkage}(X)$ 
3  $n \leftarrow \text{size}(X)$ 
4 for  $b \leftarrow 1$  to  $n$  do
5    $C_b \leftarrow \text{clustering}(b, L)$ 
6    $S_b \leftarrow \text{score}(C_b, X)$ 
7 end
8  $m \leftarrow \text{argmax}(S)$ 
9  $C = C_m$ 
/* Step III: Packing and Decomposition (Section 3.4) */
10 for  $c_i \in C$  do
11    $P_i \leftarrow \text{packing}(D[c_i])$ 
12 end
    
```

proposed TDT framework as shown in Figure 3. TDT formulates the data transformation into a byte clustering problem based on the predictability of some bytes in floating point data. As shown, TDT reorders the dataset based on a byte clustering and packs it as a transformed dataset. The transformed dataset is blocked and compressed in parallel using existing compression methods. We discuss the details of TDT and its clustering approach in Section 3 and show how it fits into a compression pipeline in Section 4.

3 TYPED DATA TRANSFORMATION (TDT)

This section explains how TDT is applied to the input data as shown in Figure 3. We first provide an overview of the TDT algorithm and then discuss the three steps of the algorithm.

3.1 Overview

The goal of TDT is to reorder data so that “similar data” segments are adjacent, thereby minimizing higher-order entropy. We define similar data segments when segments have comparable entropy or frequency characteristics. While this reordering doesn’t change the overall entropy, it significantly benefits dictionary coders by reducing higher-order entropy, such as H1 entropy [25]. Higher-order entropy finds deeper connections by using the frequency of multiple symbols, showing the potential of new patterns. Additionally, when treating these data segments with distinct entropies separately, it affects the entropy and thus helping entropy coders as well. Both of these compressors are common in generic compression tools. However, there are $d!$ possibilities of reordering a dataset with d elements. This makes the problem to be NP hard problem.

TDT uses a clustering approach to group similar data based on the observation that bytes in the same location of a float value follow a similar entropy value. By limiting the definition to groups of bytes in the same byte location in floating values, the search space reduces significantly to $n!$ possibility of clustering, where n is the number of bytes in the float type. Considering possible

reordering within clusters, clustering with reduced search space is still challenging.

Algorithm 1 shows that TDT takes dataset D as input and produces reordered dataset P in three steps to put bytes with similar entropies next to each other. The input dataset D is shown as a list of float values $(b_1, b_2, \dots, b_n)$ where b_i is the i^{th} byte in value in D and n is the word width of the type in the dataset. The first step of the TDT algorithm extracts features from each byte locations of D and creates feature vector $X = (x_1, x_2, \dots, x_n)$, where each x_i is a feature vector of byte b_i (e.g., a vector in a multi-dimensional space). Then the second step clusters the features to create the clustering $C = (C_1, C_2, \dots, C_k)$ where C_j is a byte cluster j including a set of features x_i corresponding to byte IDs b_i . Finally, the third step creates the reordered dataset by storing bytes of the same cluster next to each other and in a separate stream as a list of P . The details of these three steps are explained in the rest of this section.

Table 1: Selected Features for the clustering algorithm. H_j : Entropy of the j -th block as shown in Equation 1. N : Total number of blocks. $\text{Count}(i)$: Number of occurrences of byte value i in the entire byte group.

Feature Name	Mathematical Formulation
Average Entropy	$\frac{1}{N} \sum_{j=1}^N H_j$
Standard Deviation of Entropy	$\sqrt{\frac{1}{N-1} \sum_{j=1}^N (H_j - \text{Average Entropy})^2}$
Maximum Entropy	$\max(H_1, H_2, \dots, H_N)$
Minimum Entropy	$\min(H_1, H_2, \dots, H_N)$
Normalized Byte Frequency	$f_i = \frac{\text{Count}(i)}{\sum_{j=0}^{255} \text{Count}(j)}, i = 0, 1, \dots, 255$

3.2 Step I: Feature Extraction

While the entropies of bytes are uniform but their value changes across datasets. Therefore, using one entropy for one component across datasets does not represent it well for data transformation. To capture the complex characteristics of data for compression, the first step of the TDT algorithm uses a combination of statistical features based on entropy to calculate the similarity between bytes. The feature extraction process is designed to capture the statistical characteristics of each byte group over a fixed-size block. Compressing a block of data is common due to limited fast memory in computers. The *featureExtraction* function in line 1 in Algorithm 1 computes a set of entropy-based features from the input dataset.

The five features selected to represent information within a block of bytes are shown in Table 1. The first four features are based on entropy H , as defined in Equation 1.

$$H = - \sum_{i=1}^q p_i \log_2(p_i) \quad (1)$$

where p_i is the probability of the i -th byte value within the window, and q is the total number of unique byte values (256 for 8-bit data). These four features—average entropy, standard deviation of entropy,

maximum entropy, and minimum entropy— capture, respectively, overall randomness, inconsistencies in data randomness, and the maximum and minimum variability within the data. We also use normalized byte frequency as the only feature in the frequency category, as shown in Table 1. The normalized byte frequency of each byte value (0–255) is calculated for the entire byte group. This 256-dimensional vector captures the distribution of byte occurrences and serves as a global signature for the group.

The selected feature by the algorithm is a combination of all five features listed in Table 1, often depending on the design choice, such as compression method and dataset properties. The extracted features for each byte group include the entropy-based metrics (average, standard deviation, maximum, and minimum entropy) and the byte frequency vector. These features are concatenated into a single feature vector, providing a comprehensive statistical representation of the byte group as shown with $x_i \in \mathbb{R}^{260}$ as for the feature vector of the i th byte group:

$$x_i = [\bar{H}_i, \sigma(H_i), \max(H_i), \min(H_i), f_i(0), f_i(1), \dots, f_i(255)],$$

where components are defined based on Table 1 for a given byte group i .

3.3 Step II. Clustering

The clustering problem in the second step of the TDT algorithm takes a list of n features $X = (x_1, x_2, \dots, x_n)$ and partitions them into k clusters C . The goal is to find a clustering that maximizes the entropy difference between clusters. We define this objective as maximizing the minimum inter-cluster distance as defined in Equation 2:

$$\max_{C_1, \dots, C_k} (\min_{i \neq j} \|x_i - x_j\|) \quad (2)$$

with the constraint that each byte b_i must belong to exactly one cluster: $\bigcup_{j=1}^k C_j = X$ and $C_i \cap C_j = \emptyset$ for $i \neq j$. The number of clusters is $1 < k \leq n$. The distance between two clusters i and j is computed using Euclidean distance between the two corresponding features x_i and x_j , as shown with $\|x_i - x_j\|$.

Lines 2–7 in Algorithm 1 shows the clustering step of the TDT algorithm, where it takes the feature list of X and creates a clustering C . The clustering in Algorithm 1 evaluates clusterings with different numbers of possible clusters and selects the best one with respect to a clustering score. The algorithm starts by assuming each byte as one cluster, which means n clusters. In other words, for single-precision datasets, initially there are 4 clusters and 8 clusters for double-precision datasets. In line 2, the algorithm computes the pairwise distance between every pair of partitions and builds an $n \times n$ matrix L . This matrix is called a linkage matrix. Then, the algorithm creates clusterings and their scores for all possible cluster counts, which range from 1 to n (lines 4–7). Finally, the index of the clustering with the maximum score is selected in line 8, where argmax returns the index of the maximum element in S .

The choice of clustering method and score in Algorithm 1 is set experimentally to provide the best clustering with the maximum objective (Equation 2). We find that hierarchical clustering provides the best results. For the clustering score, TDT supports four well-established internal validation metrics: Silhouette Score [30], Davies-Bouldin Score [12], Calinski-Harabasz Score [5], and Gap

Statistic [37]. Based on the results of this evaluation, we established that for 32-bit floating-point datasets, we primarily rely on the *Davies-Bouldin Score*, and for 64-bit floating-point datasets, we consider using a *Gap Statistic Score* computed across all features.

3.4 Step III. Packing and Decomposition

Upon completion of the clustering process, the input dataset D will be reorganized according to the clustering information C . This reorganization offers several strategies for arranging bytes adjacent in the transformed dataset based on the selected inter-cluster and intra-cluster ordering.

Intra-cluster byte ordering decides the placement order of bytes within each cluster. We limit this to either group bytes from the same floating value first, i.e. *same-value packing*, or group same bytes across values, i.e. *same-byte packing*. We found the two approaches performed similarly with respect to compression ratio. We choose same-byte packing, but it can be defined by the user if needed.

Inter-cluster alignment decides how to place a packed cluster in the transformed dataset. Our findings indicate that the order in which clusters are stored has a negligible impact on the TDT objective. This observation suggests that clusters of bytes can be decomposed, and the offset to the beginning of each byte cluster can be stored. Decomposition into smaller data arrays with lower entropy offers two advantages: ① Treating each byte cluster as an independent data stream results in a lower average entropy [11] and potentially increases the compression ratio, where the selected method relies on a pure entropy-based compression technique. ② Operating on multiple independent data streams, rather than a single stream, potentially enhances throughput. This approach introduces additional workloads beyond those typically achieved through common data parallelism in compression tools, e.g., blocking. We will discuss how these components are used when combined with a compression tool.

4 TDT-BASED COMPRESSION

TDT augments generic compression tools with the knowledge of floating-point types to provide a high compression ratio. Figure 3 shows how the transformed dataset is provided as input to a blocked compression tool. This section discusses how data parallelism is used through blocking and how TDT is combined with different compression tools and algorithms. The section also discusses some TDT parameters and selects them to achieve a high compression ratio.

4.1 Parallel Blocking-based De/Compression

To efficiently integrate TDT into compression methods, this section discusses its combination with blocking to utilize parallel resources. Blocking is a common technique for exploiting data parallelism in compression methods. This technique partitions the input data into blocks, where each block can be independently de/compressed in parallel.

To combine blocking with TDT, the order in which clustering and blocking are applied is essential. Blocking can be applied to each cluster of bytes separately, or each block can be clustered independently. We investigate both options and find that applying

TDT to each block leads to better throughput with minimal effect on the compression ratio. In other words, the TDT-based compression first blocks the dataset and then applies transformation per block. This makes sense since applying TDT before chunking does not allow for interleaving data transformation and compression, where threads can be scheduled to use memory and computing resources more efficiently.

Another critical decision for combining TDT with blocking is determining the block size. A larger block size enables a larger window for the compression method, potentially leading to better compression ratios. However, larger blocks also reduce the number of parallel workloads and degrade cache efficiency when they exceed the cache size. We found that the block size should be approximately between L1 and L2 cache sizes to ensure data locality during de/compression. L1 and L2 cache sizes are dedicated to each core in CPUs, hence selected as a limit in this work. The exact size varies from one compression tool to another due to the different memory access patterns of the compression algorithms used. The block size selection also has an impact on load balance, where a larger block size leads to a smaller number of blocks, potentially impacting core utilization. Therefore, we limit the number of blocks, i.e., $\frac{\text{dataset size}}{\text{block size}}$, to be always larger than the number of cores. Additionally, all contiguous blocks should be assigned to the same thread to maximize the benefits of prefetching in the cache levels.

Method	Algorithm/Library	Architecture
Pure Foundational	LZ77	Single-Thread
	Huffman Coding	Single-Thread
	Delta-based	Single-Thread
Hybrid	zstd	Multicore CPU
	snappy	Multicore CPU
	lz4	Multicore CPU
	zlib	Multicore CPU
	bzip	Multicore CPU
	nvCOMP	GPU

Table 2: Used compression methods/libraries with TDT.

4.2 Compression Methods

To evaluate the effect of TDT on compression, this section presents the compression methods used in TDT-based compression. We use two classes of compression methods for this evaluation. We select pure foundational compression algorithms with in-house implementations to demonstrate the effect of TDT on them in isolation. Additionally, we choose generic high-performance compression tools to show the impact of TDT on the performance and compression ratio of hybrid CPU and GPU-based tools, which combine foundational methods with efficient implementations. Table 2 provides an overview of the two categories of compression methods studied in this work. This section briefly discusses each category.

4.2.1 Pure foundational methods. As shown in Table 2, we select three important compression algorithms to study the effect of TDT in isolation for each of them. The three algorithms are: ① The LZ77 algorithm is a general dictionary-based method, known as a key technique to reduce file sizes by spotting and removing repeated data patterns. LZ77 works by maintaining a moving window over the data being compressed. When it finds a sequence of characters that appears more than once within this window, it replaces the extra copies with a reference to the first instance. ② The Huffman coding [16] algorithm is an entropy-based method that assigns a shorter codeword to more frequent symbols. Data transformation does not affect entropy. Therefore, data transformation without decomposing byte clusters does not affect the compression ratio of Huffman codes. ③ Delta-based methods examine the differences between consecutive floating-point data to exploit redundancy and compress the data. These methods are often used for streaming data when only part of the data is accessible. We use an XOR-based scheme similar to Gorilla [28] for this algorithm.

4.2.2 Hybrid methods. State-of-the-art compression packages often combine different compression algorithms and run them efficiently on parallel CPU cores and vector units. As shown in Table 2, we study zstd [13], snappy [15], and lz4 [8], zlib [14], and bzip [34] working on multicore CPUs and nvCOMP-LZ4 [10] for GPUs as representatives of efficient combinations of dictionary and entropy coders. The selected compression tools are a combination of run-length encoding (RLE) and a dictionary-based coding and/or an entropy coder such as Huffman coding [16] and benefit from common data transformation.

These selected compression tools also have different performance throughput and compression ratios on multicore and GPUs. We provide a short summary of each tool: ① zstd uses LZ77 algorithm followed by Finite State Entropy (FSE) [13] and Huffman Coding which are entropy coders. FSE is an entropy coder that encodes and decodes processes based on transitions between a finite number of states. These states hold information about the probability distribution of the symbols encountered so far. zstd provides different levels of compression to ensure a compromise between compression ratio and throughput. ② snappy is a compression method based on dictionary-coding, similar to LZ77. snappy focuses on throughput by reducing the window search for finding patterns and using a simple greedy approach. ③ lz4 is also following a similar scheme to snappy, with more emphasis on throughput. ④ zlib implements the DEFLATE algorithm [27] which is a combination of LZ77 and Huffman coding. ⑤ bzip combines RLE, BWT [25], move-to-front transform [3], and Huffman coding which degrades its throughput. ⑥ nvCOMP is a vendor library provided by NVIDIA and is commonly used to compress data using a hybrid of foundational compression methods. nvCOMP has implementations of several compression algorithms. We use lz4 variant of nvCOMP for fair comparison with its CPU variant where needed.

4.3 Implementation

This section discusses some implementation details to reduce clustering overhead using compression modes and to encode the TDT transformation details in the compressed data.

4.3.1 Compression Modes. To avoid the cost of clustering during compression, we provide two modes for TDT-based compression: dynamic and static modes. The dynamic mode relies on clustering for each dataset in the compression phase. For when the size of the data is big, the cost of clustering amortizes over the compression time. We apply clustering on a portion of the dataset. We find that 30% of the dataset is sufficient to capture the properties of the dataset. The static mode relies on clustering information based on clustering per application. This clustering information is collected through extensive profiling in our datasets from several domains. To calculate clustering in static mode, datasets are first categorized by application and their type width. Then, a sample of each dataset is merged to make a compound dataset. The clustering is then applied to the compound dataset and used for all datasets of the category. These two modes provide a trade-off between compression throughput and compression ratio in TDT-based compression.

4.3.2 Compression format. The TDT-based compression encodes all metadata in the compressed data with small overhead. The metadata encoded in the compressed dataset is the clustering information, selected compression methods, and block size. The compression tool is encoded as an integer ID. The compressed blocks and byte clusters are stored with their offset, e.g., as a 2D array with variable length dimensions, so accessing to each compressed part in a block is easily accessible, essential for parallel decompression. So during decompression, first the compression tool is encoded, and then block offsets and byte cluster offsets are used to locate compressed data and then decompress them in parallel.

5 EXPERIMENTAL RESULTS

This section evaluates the efficiency of TDT-based compression using a wide spectrum of datasets and compression tools. Particularly, this section answers the following fundamental questions: ❶ What is the effect of TDT on compression ratios of different foundational and hybrid compression methods across domains? ❷ How does TDT affect throughput of compression and decompression when combined with other methods in CPU and GPUs? ❸ How are TDT critical components, such as clustering method, feature selection, and blocking selected?

5.1 Setup

5.1.1 Dataset. The list of datasets used for TDT is provided in Table 3. We apply TDT to a list of datasets from different applications obtained from FCbench [6]. The selected datasets are from HPC [19, 36], TS [33], OBS [31], and DB [39]. While selected FCbench datasets have both double precision (D or float64) and single precision (S or float32), it does not have half-precision. We use a trained language model, Llama 7B [38], and extract its weights and attention matrices with half precision (H) and store them all as one big dataset.

5.1.2 Platform and Measurements. We evaluate our method on both CPU and GPU architectures. We select a consumer-grade CPU and GPU for this purpose. The CPU is *Core-i9* with 16 cores, 760 KB, 24MB, and 30MB L1/L2/L3 cache sizes. We use NVIDIA GeForce RTX 4090 with Ampere architecture. The GPU has 3584 Cuda cores with L1/L2 cache sizes of 128 KB per SM and 3 MB, respectively. A

ID	Application	Type (S/D)	Size (bytes)	Entropy	Domain
D1	astro-mhd	D	548M	0.97	HPC
D2	rsim	S	94M	18.50	HPC
D3	turbulence	S	67M	23.73	HPC
D4	wave	S	537M	25.27	HPC
D5	num-brain	D	142M	23.97	HPC
D6	num-control	D	160M	24.14	HPC
D7	astro-pt	D	671M	26.32	HPC
D8	msg-bt	D	266M	23.67	HPC
D9	citytemp	S	12M	9.43	TS
D10	wesad-chest	D	272M	13.85	TS
D11	solar-wind	S	424M	14.06	TS
D12	hdr-night	S	537M	9.03	OBS
D13	hdr-palermo	S	843M	9.34	OBS
D14	hst-wfc3-ir	S	24M	15.04	OBS
D15	hst-wfc3-uvis	S	109M	15.61	OBS
D16	acs-wht	S	225M	20.13	OBS
D17	spitzer-irac	S	165M	20.54	OBS
D18	jws-mirimage	S	169M	23.16	OBS
D19	tpcxBB-store	D	790M	16.73	DB
D20	tpcxBB-web	D	987M	17.64	DB
D21	tpcH-order	D	120M	23.40	DB
D22	tpcDS-catalog	S	173M	17.34	DB
D23	tpcDS-store	S	277M	15.17	DB
D24	tpcDS-web	S	86M	17.33	DB
D25	Llama [38]	H	48G	4.88	ML

Table 3: Combined Datasets from HPC, TS, OBS, and DB Domains with Application and ID Columns (*S/D denotes single-/double-precision).

C++ benchmark is created to drive all compression tools and TDT for fair comparison. The benchmark is built using CMake and GCC compiler version 11.4.0. All experiments are performed on Ubuntu 22.04.3 LTS. We also implemented TDT in Python to verify the compression ratio with Python libraries of the compression tools. For parallel implementations of TDT, OpenMP is selected for CPUs and Cuda is used for GPU where needed. A close thread binding is selected to ensure each thread is mapped to a physical core.

To assess compression performance, we use three key metrics: *compression ratio (CR)*, *compression throughput (CT)*, and *decompression throughput (DT)*. These are defined as $CR = \frac{\text{Original Size}}{\text{Compressed Size}}$, $CT = \frac{\text{Original Size}}{\text{Compression Time}}$, and $DT = \frac{\text{Original Size}}{\text{Decompression Time}}$, consistent with prior work [6]. In addition to these absolute metrics, we introduce *compression ratio improvement (CRI)* as a relative measure. CRI quantifies the effect of our approach’s (specifically, TDT’s) compression ratio compared to a baseline, defined as $CRI = \frac{\text{TDT CR}}{\text{Baseline CR}}$. We similarly define a relative throughput improvement metric as compression throughput (CTI) and decompression throughput (DTI).

5.1.3 Compression tools. TDT is combined with compression tools provided in Table 2. Foundational methods, except for LZ77, are from an in-house proof-of-concept, only for compression ratio analysis. Due to the inefficiency of these implementations, they are excluded from throughput evaluation. For LZ77, we use the latest

ID	Dataset	zstd		snappy		lz4		zlib		bzip2		nvCOMP	
		TDT	Standard	TDT	Standard	TDT	Standard	TDT	Standard	TDT	Standard	TDT	Standard
D1	astro_mhd	33.146	29.871	12.934	12.250	28.540	25.216	30.711	27.753	28.621	25.216	18.910	22.670
D2	rsim	1.805	1.483	1.638	1.297	1.745	1.358	1.745	1.451	1.782	1.440	1.580	1.310
D3	turbulence	1.252	1.093	1.199	1.000	1.209	0.996	1.264	1.092	1.285	1.063	1.130	1.000
D4	wave	1.794	1.253	1.453	1.000	1.358	1.077	1.676	1.154	1.421	1.164	1.280	1.030
D5	num_brain	1.265	1.065	1.209	1.000	1.234	0.997	1.262	1.064	1.241	0.997	1.200	1.000
D6	num_control	1.146	1.058	1.100	1.010	1.107	1.013	1.154	1.057	1.154	1.030	1.020	1.010
D7	astro_pt	1.366	1.047	1.262	1.000	1.320	0.999	1.330	1.048	1.331	0.999	1.260	1.000
D8	msg_bt	1.383	1.126	1.277	1.052	1.318	1.065	1.345	1.131	1.324	1.065	1.270	1.060
D9	citytemp	2.987	2.665	2.023	1.617	2.013	1.974	3.117	2.530	3.589	3.882	1.600	1.370
D10	wesad_chest	3.305	3.349	2.257	2.351	2.785	2.953	3.563	3.344	2.810	2.953	1.810	2.130
D11	solar_wind	1.708	1.440	1.375	1.111	1.491	1.343	1.811	1.582	1.998	2.021	1.290	1.170
D12	hdr_night	3.647	2.842	2.855	1.832	3.176	2.178	3.679	2.716	3.822	3.977	2.700	1.400
D13	hdr_palermo	4.329	3.451	3.304	1.947	4.000	2.391	4.437	3.196	5.647	5.598	3.250	1.420
D14	hst_wfc3_ir	2.011	1.773	1.867	1.478	1.962	1.526	2.014	1.761	2.105	1.743		
D15	hst_wfc3_uvis	1.935	1.756	1.749	1.555	1.839	1.610	1.935	1.749	1.955	1.702	1.710	1.540
D16	acs_wht	1.580	1.369	1.437	1.142	1.478	1.166	1.586	1.362	1.612	1.429	1.410	1.160
D17	spitzer_irac	1.174	1.076	1.104	1.000	1.110	0.996	1.178	1.074	1.180	1.044	1.050	1.000
D18	jw_mirimage	1.313	1.148	1.287	1.000	1.299	0.999	1.322	1.145	1.318	1.150	1.270	1.000
D19	tpcxb_store	2.561	2.333	1.878	1.644	2.204	2.028	2.625	2.470	2.170	3.130	1.530	1.730
D20	tpcxb_web	2.485	2.224	1.830	1.611	2.167	1.986	2.548	2.382	2.110	2.960	1.510	1.690
D21	tpch_order	2.091	1.836	1.502	1.359	1.659	1.598	2.113	1.864	1.662	1.598	1.220	1.500
D22	tpcds_catalog	1.460	1.238	1.156	1.057	1.229	1.113	1.440	1.301	1.474	1.520	1.110	1.060
D23	tpcds_store	1.554	1.353	1.248	1.155	1.370	1.253	1.610	1.450	1.800	1.713	1.130	1.110
D24	tpcds_web	1.461	1.238	1.156	1.057	1.230	1.113	1.441	1.301	1.475	1.520	1.110	1.060
D25	LLama	1.912	1.788	1.426	1.071	1.399	1.429	1.970	1.743	1.910	2.090		

Table 4: Compression ratio comparison (TDT vs. standard)

version of fastlz¹, which an efficient single-threaded implementation of the LZ77 algorithm. We use bzip v1.0.8, lz4 v1.9.4, nvComp v4.1.1.1, and the latest release of snappy, zlib, and zstd for comparison. Public repositories of bzip², lz4³, snappy⁴, zlib⁵, zstd⁶ are used to obtain the latest version or release of compression tools. NVComp is a proprietary tool, and we used the CUDA toolkit version 11 for comparison. All compression tools are used with their default configurations. All numbers are reported TDT static mode unless otherwise stated.

5.2 Compression Ratio Evaluation

This section presents how TDT-based compression performs across datasets and tools.

5.2.1 Overall comparison. Figure 4 shows an overall view of the distribution of compression ratios for each hybrid compression tool. The x-axis shows the compression tools, and the y-axis shows the compression ratio across all datasets in Table 3 for the tool. As shown, TDT improves the compression ratio of all tools across datasets. On average, TDT provides between 1.04–1.20× geometric mean CRI across tools. Table 4 shows the exact compression ratio for each dataset and compression tool. In the rest of this section, we analyze these data from different perspectives.

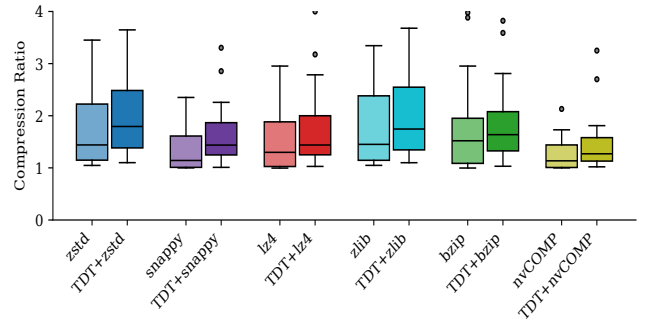


Figure 4: TDT improves CR across datasets. Some outlier points are cropped for better illustration.

5.2.2 Effect of TDT on foundational methods. Figure 5 shows the effect of TDT improves compression ratios on different foundational methods discussed in Table 2. As shown in Figure 5, the compression ratio for all foundational methods increases. The CRI for TDT for LZ77, Huffman, and Delta-based methods are 1.20, 1.14, and 1.35×, respectively. TDT benefits LZ77 more than Huffman as it takes advantage of both reordering and decomposition.

5.2.3 Effect of TDT on hybrid methods. Figure 6 illustrates how TDT improves compression ratios of datasets across compression tools. While there is consistent improvement across all datasets and tools, the effect of the transformation varies across tools. For

¹<https://github.com/ariya/FastLZ>

²<https://gitlab.com/bzip2/bzip2>

³<https://github.com/lz4/lz4>

⁴<https://github.com/google/snappy>

⁵<https://github.com/madler/zlib>

⁶<https://github.com/facebook/zstd>

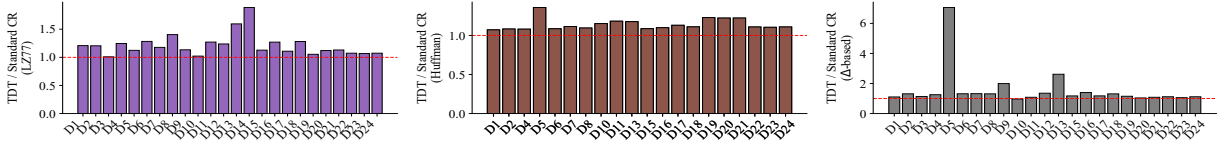


Figure 5: TDT improves compression ratio of foundational methods (LZ77, Huffman, Delta-based)

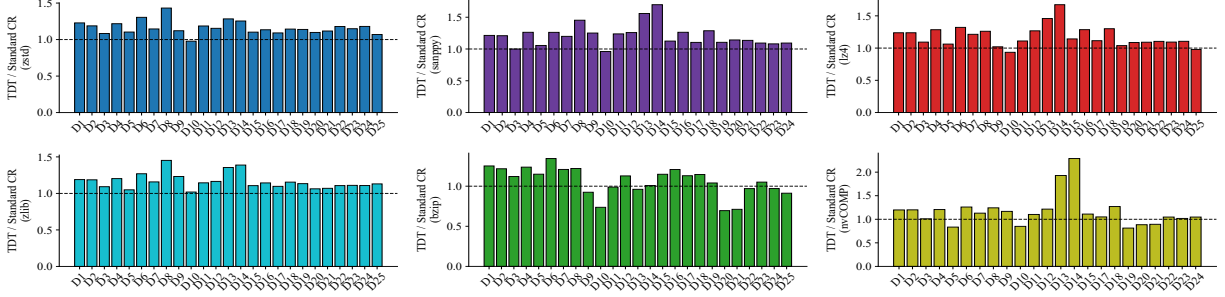


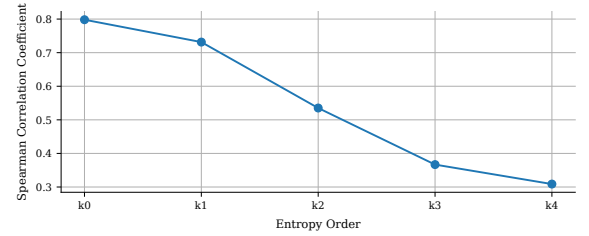
Figure 6: TDT Compression Ratio vs. standard Compression ratio Across Datasets for zstd, snappy, lz4, zlib, bzip, and nvCOMP.

bzip, the CRI is lower than for other tools. This is attributed to the internal BWT data transformation used in bzip, which reduces the effect of TDT. However, other compression methods exhibit similar behavior due to the combination of compression categories, i.e., dictionary-based and entropy coders.

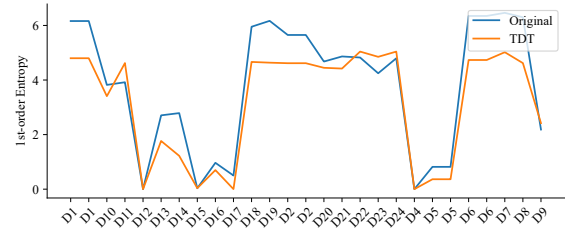
Datasets exhibit different properties that lead to different CRI when TDT is used. For example, datasets 4 and 6 consistently show the highest CR for all five compression methods. We find these datasets have the highest entropy, larger than 23, which restricts existing tools from compressing them. After applying TDT, the opportunities for compression are enabled and utilized, leading to higher CRI.

5.2.4 Entropy Analysis. The main cause for improving CR is that TDT improves the k^{th} order entropy. We compute the k^{th} order entropy for $k = 0, 1, 2, 3, 4$ and observe that while zero-order entropy is the same across datasets after TDT, the effect of TDT is noticeable in higher orders. Figure 7a shows the Spearman Correlation Coefficient between the compressed size of datasets using TDT and zstd across different entropy orders. Coefficients closer to one indicate a better correlation. As shown, there is a strong correlation for zero and first-order entropy and a moderate correlation for second-order entropy. Therefore, we use first and second-order entropy information to show the effect of TDT. Figure 7b shows that the first-order entropy of the datasets after transforming with TDT is almost always lower than the original dataset's first-order entropy. This explains why TDT improves CR across datasets.

5.2.5 Effect of TDT on application categories. As depicted in Figure 8, TDT performs differently across various application categories. Datasets from different applications provide a different range of entropy values. As shown in Table 3, time-series datasets have the lowest entropy values, while DB, OBS, and HPC have relatively higher entropy values than time-series datasets. Certain



(a) Spearman correlation coefficient between the zstd compressed size after TDT with different entropy orders of datasets



(b) Actual 1st-order entropy for all datasets for zstd

Figure 7: Entropy Analysis

compression tools also perform better with specific applications, such as zlib, which performs better when applied to DB datasets.

5.2.6 Comparing with prior floating point compressions. Figure 9 and 11 compare TDT with existing data transformations for floating-point data, specifically bitshuffle [26] and a floating point compression tool fpzip. bitshuffle and fpzip perform the best comparing with other tools such as pFPC [4] and SPD [7], hence selected for comparison. We use existing implementations of bitshuffle⁷. For

⁷<https://github.com/h5py/h5py>

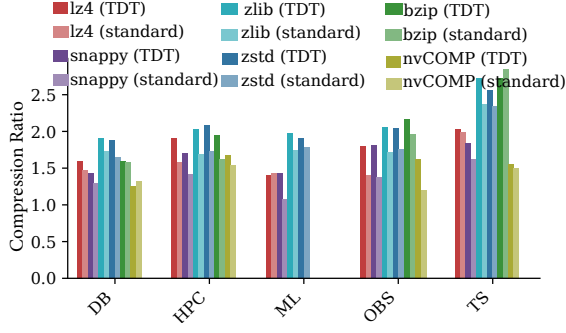


Figure 8: TDT compression ratio across applications.

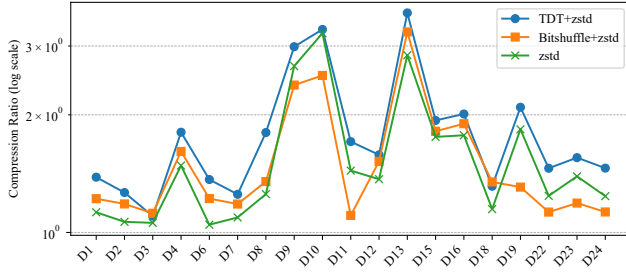


Figure 9: bitshuffle compression ratio across datasets

a fair comparison, the CR values in Figure 9 are reported when both data transformations are used with the zstd. As shown, TDT performs better than the bitshuffle data transformations across all applications. The GMean CRI of TDT over bitshuffle is 1.24×

Figure 11 compares GMean CR for different variants of TDT-based with fpzip. Although fpzip is tuned specifically for floating-point streams, combining TDT with a generic compressor still yields consistent wins. In datasets with nonuniform value distributions or repeated patterns, TDT’s block-rearrangement step produces more homogeneous chunks, allowing the compressor to capture redundancy that fpzip’s predictive model alone cannot. In addition to a higher GMean, TDT variants provide a better CR over fpzip in over 99% of datasets.

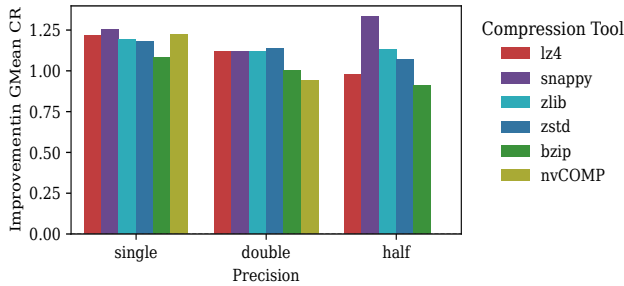


Figure 10: TDT compression ratio across different precisions

5.2.7 Floating-point type width effect. Figure 10 shows the effect of TDT on floating-point datasets with double, single, and half

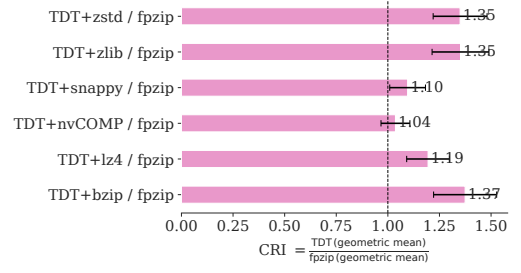


Figure 11: comparing CR with fpzip

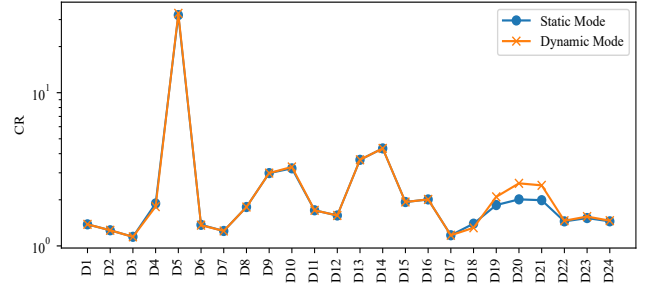


Figure 12: Effect of Dynamic vs. Statistical Mode on Compression Ratio (log scale) using zstd.

precision. The GMean CRI for double, single, and half-precision is 1.075, 1.19, and 1.08, respectively, across hybrid methods and all datasets. As shown, this trend is consistent across compression tools. Additionally, for half-precision datasets, the clustering will have only one option, which is two clusters, where each has one byte. While simple, TDT improves the compression ratio in half-precision datasets across tools.

5.2.8 Compression modes. As discussed in Section 4.3.1, TDT-based compression has dynamic and static modes. The compression ratios of the two modes when the compression tool is zstd are shown in Figure 12. As shown 12, the compression ratio across datasets follows a similar trend, indicating the efficiency of the static mode, causing zero overhead. However, dynamic mode has its use case when the data pattern is unknown.

5.3 Throughput Evaluation

This section discusses the effect of TDT on the compression and decompression throughput when combined with hybrid compression tools shown in Table 2.

5.3.1 Overall. Figure 13 provides an overview of the effect of TDT on the compression and decompression throughput when combined with different compression tools. As shown, when combined with TDT, the throughput of both compression and decompression is improved on average in all CPU and GPU methods, with GMean CTI ranging from 0.97× to 6.79× and GMean DTI ranging from 0.59× to 4.91× when using TDT compared to the standard baseline. The improvement is due to the packing and decomposition step of

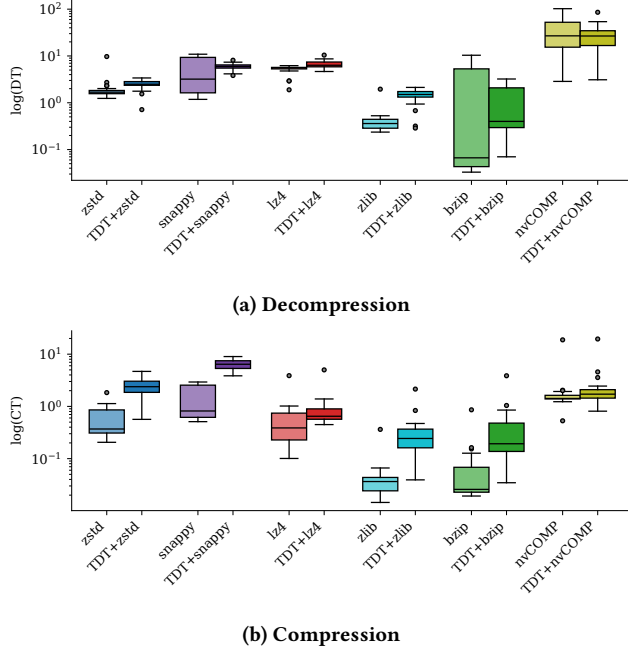


Figure 13: TDT throughput in GB/s (some outlier points are cropped for better illustration)

TDT, which creates more independent workloads for each block, enhancing parallelism and load balancing.

As shown in Figure 13, the compression throughput is lower than the decompression throughput in all tools. The GMean compression and decompression throughput improvement are 0.96 $\times$ and 3.09 $\times$, respectively, indicating a more significant effect of TDT on the compression throughput. One reason for the difference is that the compression phase in dictionary-based compression, which is common in all hybrid methods except for bzip, takes more computation, and packing enables more parallelism and better use of resources. The de/compression throughput follows a different trend in bzip under TDT because the method relies on memory-bound operations, and packing and parallelism do not help as much as with other compression techniques.

5.3.2 CPU vs GPU efficiency. The throughput of GPU methods is better than CPU methods thanks to their extensive computing cores. TDT improves the GPU compression and decompression throughput by a GMean of 1.93 $\times$ and 24.49 $\times$, respectively. This indicates TDT effectiveness on GPUs as well as CPUs.

5.4 TDT Empirical Design Choices

TDT and its compression pipeline are based on some parameters that are selected empirically. This section discusses the design choices in TDT and the TDT-based compression pipeline which are feature selection (Section 3.2), clustering parameters (Section 3.3), packing (Section 3.4), and block size (Section 4.1).

5.4.1 Clustering objective. To justify the effectiveness of the clustering objective in Equation 2, we provide two correlation plots

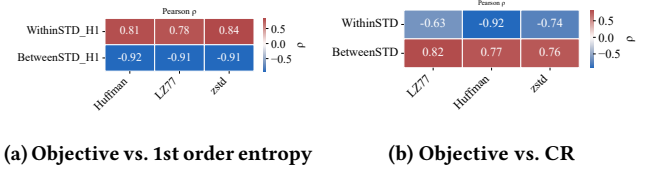


Figure 14: The choice of the clustering objective

in Figure 14. Figure 14a shows a high correlation between the objective and reducing the first-order entropy deviation intra-cluster and increasing the first-order entropy deviation between clusters. This complies with the objective of TDT to improve the first-order entropy. Also, even though Equation 2 focused on between clusters, the deviation within clusters is minimized. This entropy minimization within clusters is also aligned with the CR improvement objective, as Figure 14b provides a good correlation with CR.

5.4.2 Feature selection and clustering metric. The first step of TDT is to extract features from each initial byte cluster, later used for clustering. We select a combination of features in Table 1. We tailor the feature selection with a compression pipeline and select features and clustering metrics that provide the highest compression ratio. To do this, datasets are separated into two categories based on their precision: float32 and float64 datasets. Then we assessed three feature sets—combined entropy features, frequency, and a combination of both—by measuring their compression ratios. For each choice of feature, four different clustering metrics (Silhouette, Davies-Bouldin, Calinski-Harabasz, and Gap Statistic) are used to cluster the input feature for all datasets. This makes a search space with 12 different possibilities. We report the GMean compression ratio across all float32 and float64 datasets as shown in Figures 15a and 15b, respectively.

As shown in Figure 15, the optimal feature and clustering metric is different for float32 and float64 datasets. As shown in Figure 15a, the frequency feature, when combined with the Davies-Bouldin metric, provides the highest compression ratio, thus selected for float32 datasets. The selected feature and clustering metric for float64 datasets are combined features and Gap Statistic due to the best compression ratio as shown in Figure 15b. We saw a similar trend in other compression tool and thus analysis for zstd is shown as a representative tool.

5.4.3 Clustering algorithm efficiency. We discuss the efficiency of TDT clustering from how it picks the optimal point without testing all clustering possibilities and testing all values in the dataset. To avoid testing all possible clustering, the second step of TDT (Algorithm 1) only tests 4 and 8 possibilities for float32 and float64 datasets. To show the efficiency of the algorithm, we create an exhaustive search algorithm that tests all clustering possibilities for float32 and selects the clustering with the maximum compression ratio as an optimal solution. Then the result of TDT clustering and the exhaustive search algorithm is shown in Figure 16. As shown, the TDT clustering step is producing the optimal clustering in 95% of the datasets and for the remaining it provides a close to optimal clustering. This indicates the efficiency of the clustering step of the TDT algorithm.

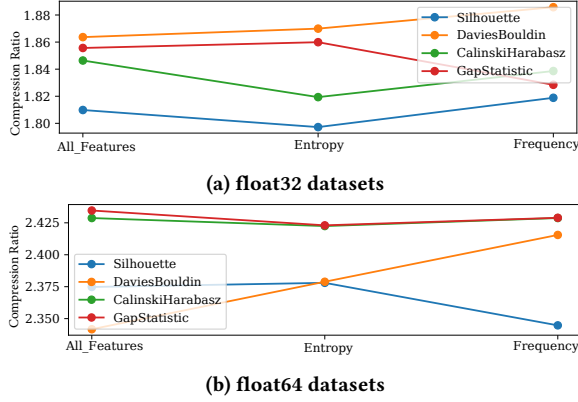


Figure 15: The effect of feature and clustering score on CR.

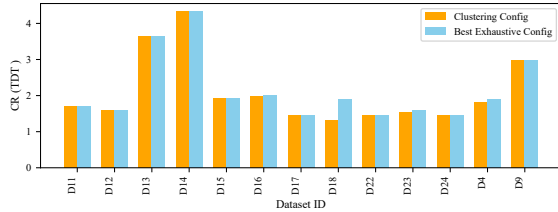


Figure 16: TDT clustering vs exhaustive search using zstd

As discussed in Section 4.3.1, 30% of values in a dataset represent characteristics of the full dataset. This approximation reduces the feature extraction time. We conducted two experiments to ensure this approximation does not affect the result. We first redo Figure 15 using 30% datasets, and the outcome is similar in terms of feature and clustering metric selection. Then we additionally used the selected feature and clustering metric from 30% of datasets and computed the resulting compression ratio and we found that the compression ratio remains the same in 92% of datasets. Therefore, we select 30% of values to reduce computation time for clustering.

5.4.4 Packing Decision. As discussed in Section 3.4, byte-clusters can be packed to put bytes from the same value next to each other (same-value) or all bytes in the same byte location (same-byte). Since both packing approaches have their advantage, we select the packing approach that provides the highest compression ratio across datasets. We measured both packing approaches across a given clustering and computed the resulting compression ratio across datasets. The compression ratios of datasets were similar in all but two datasets, where same-byte packing performed better, hence the default choice in TDT.

5.4.5 Block-size selection. As discussed in Section 4.1, the block size is selected to use the memory hierarchy of CPU and GPU processors and minimize the effect of load balance and compression ratio. We tune all block sizes between L1 cache and last level cache (LLC), which is L3 in the CPU, to decide about the best block size. For GPU, since the block size is set internally by the tool, we do not specify a block size. To isolate the effect of threading from

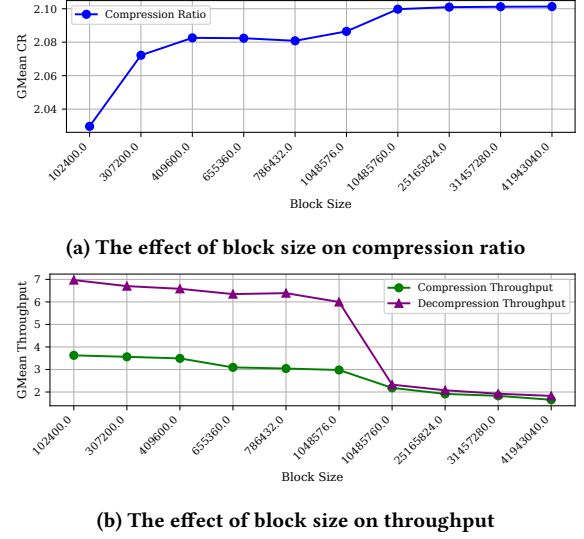


Figure 17: Block size trade-off for Fastlz

internal implementation, we use fastlz, which is a single-threaded open-source compression tool. We use blocking to make the tool parallel with the sweep of the block size.

Figure 17a shows the effect of block size on compression ratio for the selected block range sweep across all datasets. As shown, the compression ratio for block sizes larger than L1 changes between 2.08–2.1, which is insignificant. Therefore, the goal of blocking is to select a block size that maximizes throughput within the range that has minimal effect on compression ratio. As shown in Figure 17b, varying block size drastically affects throughput in both compression and decompression. As shown in Figure 17b, between L1 and L2, the throughput is close to the peak throughput. Therefore, we select a block size between L1 and L2 for each compression tool for a good balance between compression ratio and throughput.

5.4.6 Packing and decomposition interaction with blocking. As discussed in Section 4.1, the order in which blocking and packing are applied does affect throughput. The selected order in TDT is blocking, followed by packing and decomposition. Figure 18 shows the effect of this decision on compression and decompression throughput. As shown, blocking followed by packing leads to a 4.08 and 1.42 improvement over the standard approach in compression and decompression throughput, respectively, while packing followed by blocking results in a 1.22 improvement in compression throughput and a 0.61 slowdown in decompression throughput. The main reason behind this is that blocking can be done without copying overhead, while packing needs explicit copying, as generic compression tools need contiguous memory locations. Therefore, when blocking comes first, the copy overhead packing can be done in parallel.

5.5 Discussion and potential extensions

This section discusses an initial study of the potential extension of TDT for other primitive types such as integers. We applied TDT

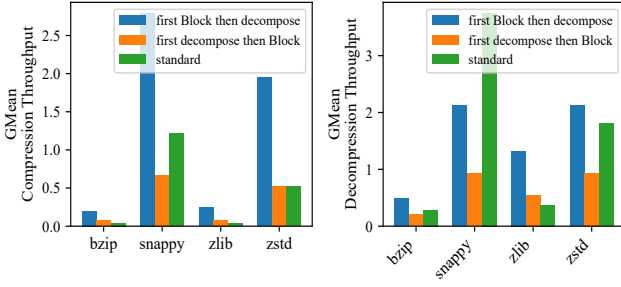


Figure 18: Decomposition and blocking order on Throughput (GB/s)

to two integer datasets, Poker-Hand⁸ and Coverttype⁹ for two representative compression tools as shown in Table 5. While the study is limited but the result indicates strong potentials in other typed datasets such as integers. TDT provides a CRI between 1.08–1.67× across these two datasets and compression tools.

Dataset	zstd		bzip2	
	TDT	Standard	TDT	Standard
poker-hand (32 bits)	7.25	4.35	8.29	7.26
coverttype (64 bits)	32.73	23.14	40.03	36.75

Table 5: Compression ratio for integer datasets.

6 RELATED WORK

6.1 Compression methods

We classify lossless compression methods into general and floating-based methods. Common general compression tools treat floating-point data as bytes and compress it using a combination of compression techniques. Some notable compression tools in this category are zstd [13], snappy [15], lz4 [8], zlib [14], and bzip [34]. These compression methods treat floating-point data as bytes and compress them. While effective, these tools do not take advantage of the characteristics of floating-point values.

Using floating-point standards or domain information to compress floating-point datasets, e.g., in scientific workloads [22] and checkpointing data [35], is well explored area [6]. Lossy compression methods such as ALP [1] and BUFF [24] reduce the precision of the values and apply compression methods. While effective, removing precision is not acceptable in several domains such as medical imaging and machine learning models. Lossless compression methods often rely on a smooth transition of data in streaming applications and use a prediction-based method to compress the data. The compression methods either use a prediction-based model [4, 18, 41] or rely on previous values [20, 21, 28] to predict the next value. The smooth pattern in streaming applications creates similar bit patterns in part of floating types, when applying

an XOR on consecutive values, resulting in leading or trailing zeros, used for compressing in Gorilla [28], Chimp [21], Elf [20], and other streaming methods [29]. The other category of floating-point compression uses compression techniques such as prediction-based compression algorithms [23], dictionary-based methods [7].

6.2 Data transformation

Data transformations are commonly applied to various datasets to improve the compression ratios. One common data transformation is the Burrows-Wheeler transform (BWT) [25] which is typically applied to strings such as in genetic data. BWT is also used in general compression tools such as bzip [34]. While efficient for strings, they do not take advantage of floating-point value standards characteristics. Another potentially relevant work is the clustering approaches used for compression in database applications such as [40]. While the partitioning approach in [40] shares common elements, in floating-point datasets, it is also important to ensure high deviation between clusters, necessary for entropy coders.

Data transformations are applied to floating datasets to improve the compression ratio. Brotli [2] clusters data to group data with a similar nature in one cluster. The clustering scheme in Brotli is coarse and does not differentiate the different entropies of bytes in floating-point datasets. One of the closely related works is shuffle [9] and Bitshuffle [26]. These data transformation approaches reorganized the floating-point data, such as float, at the bit level to create more repetitive patterns in imaging data. While these approached use floating-point properties but they do not consider the effect of bytes together, which differ across datasets. TDT transforms floating point datasets based on dataset characteristics and improve compression ratios across applications and compression tools.

7 CONCLUSION AND FUTURE WORK

Lossless compression of floating-point values is critical for several domains. Prior general and floating-point-based compression methods effectively compress floating-point datasets. However, there is still potential to improve compression methods by considering the characteristics of floating-point data. This paper proposes TDT, a technique that uses the non-uniform entropy distribution of different bytes in floating-point datasets to transform the data and create a more uniform dataset. This data transformation is mapped to a clustering problem and implemented in a compression pipeline. As a result, the compression ratio are improved across tools between 1.04–1.17× and the compression throughput is improved between 1.16–7.57×. We plan to test this approach for other types of numerical values, such as integers, and evaluate its effectiveness for lossy methods.

ACKNOWLEDGMENTS

This work is supported by NSERC discovery grants (RGPIN-2023-04897, DGEER-2023-00133), NSERC alliance grant (ALLRP 586319-23), and the Digital Research Alliance of Canada (www.alliancecan.ca).

REFERENCES

- [1] Azim Afrozeh, Leonardo X Kuffo, and Peter Boncz. 2023. ALP: Adaptive Lossless floating-Point Compression. *Proceedings of the ACM on Management of Data* 1, 4 (2023), 1–26.

⁸<https://www.kaggle.com/datasets/camillahorne/poker-hands>

⁹<https://www.kaggle.com/datasets/zsinghrahulk/coverttype-forest-cover-types/data>

- [2] Jyrki Alakuijala, Andrea Farruggia, Paolo Ferragina, Eugene Kliuchnikov, Robert Obryk, Zoltan Szabadka, and Lode Vandevenne. 2018. Brotli: A general-purpose data compressor. *ACM Transactions on Information Systems (TOIS)* 37, 1 (2018), 1–30.
- [3] Ziya Arnavut. 2000. Move-to-front and inversion coding. In *Proceedings DCC 2000. Data Compression Conference*. IEEE, 193–202.
- [4] Martin Burtscher and Paruj Ratanaworabhan. 2008. FPC: A high-speed compressor for double-precision floating-point data. *IEEE transactions on computers* 58, 1 (2008), 18–31.
- [5] Tadeusz Caliński and Jerzy Harabasz. 1974. A dendrite method for cluster analysis. *Communications in Statistics-theory and Methods* 3, 1 (1974), 1–27.
- [6] Xinyu Chen, Jiannan Tian, Ian Beaver, Cynthia Freeman, Yan Yan, Jianguo Wang, and Dingwen Tao. 2024. FCBench: Cross-Domain Benchmarking of Lossless Compression for Floating-Point Data. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1418–1431.
- [7] Steven Claggett, Sahar Azimi, and Martin Burtscher. 2018. SPDP: An automatically synthesized lossless compression algorithm for floating-point data. In *2018 data compression conference*. IEEE, 335–344.
- [8] Yann Collet. 2011. *lz4/lz4: LZ4 compression algorithm*. <https://github.com/lz4/lz4>
- [9] Andrew Collette. 2013. *Python and HDF5: unlocking scientific data*. " O'Reilly Media, Inc."
- [10] NVIDIA Corporation. 2024. nvCOMP: High-Performance GPU Compression Library. <https://developer.nvidia.com/nvcomp>. Accessed May 1, 2025.
- [11] Thomas M Cover and Joy A Thomas. 2006. *Elements of information theory*. John Wiley & Sons.
- [12] David L Davies and Donald W Bouldin. 2009. A cluster separation measure. *IEEE transactions on pattern analysis and machine intelligence* 2 (2009), 224–227.
- [13] Facebook. 1999. *Zstandard – fast real-time compression algorithm*. <https://github.com/facebook/zstd%7D>
- [14] Jean-loup Gailly and Mark Adler. 2004. Zlib compression library. (2004).
- [15] Google. 2025. *Snappy: A fast compressor/decompressor*. <https://google.github.io/snappy/>
- [16] David A Huffman. 1952. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE* 40, 9 (1952), 1098–1101.
- [17] IEEE. 2019. *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019)*. <https://standards.ieee.org/standard/754-2019.html>
- [18] Søren Kejser Jensen, Torben Bach Pedersen, and Christian Thomsen. 2018. Mod-laradb: Modular model-based time series management with spark and cassandra. *Proceedings of the VLDB Endowment* 11, 11 (2018), 1688–1701.
- [19] R Kissmann, K Reitberger, O Reimer, A Reimer, and E Grimaldo. 2016. Colliding-wind binaries with strong magnetic fields. *The Astrophysical Journal* 831, 2 (2016), 121.
- [20] Ruiyuan Li, Zheng Li, Yi Wu, Chao Chen, and Yu Zheng. 2023. Elf: Erasing-based lossless floating-point compression. *Proceedings of the VLDB Endowment* 16, 7 (2023), 1763–1776.
- [21] Panagiotis Liakos, Katia Papakonstantinou, and Yannis Kotidis. 2022. Chimp: efficient lossless floating point compression for time series databases. *Proceedings of the VLDB Endowment* 15, 11 (2022), 3058–3070.
- [22] Peter Lindstrom. 2017. *Error distributions of lossy floating-point compressors*. Technical Report. Lawrence Livermore National Lab.(LLNL), Livermore, CA (United States).
- [23] Peter Lindstrom and Martin Isenburg. 2006. Fast and efficient compression of floating-point data. *IEEE transactions on visualization and computer graphics* 12, 5 (2006), 1245–1250.
- [24] Chunwei Liu, Hao Jiang, John Paparrizos, and Aaron J Elmore. 2021. Decomposed bounded floats for fast compression and queries. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2586–2598.
- [25] Giovanni Manzini. 2001. An analysis of the Burrows–Wheeler transform. *Journal of the ACM (JACM)* 48, 3 (2001), 407–430.
- [26] Kiyoshi Masui, Mandana Amiri, Liam Connor, Meiling Deng, Mateus Fandino, Carolin Höfer, Mark Halpern, David Hanna, Adam D Hincks, Gary Hinshaw, et al. 2015. A compression scheme for radio data in high performance computing. *Astronomy and Computing* 12 (2015), 181–190.
- [27] Savan Oswal, Anjali Singh, and Kirthi Kumari. 2016. Deflate compression algorithm. *International Journal of Engineering Research and General Science* 4, 1 (2016), 430–436.
- [28] Tuomas Pelkonen, Scott Franklin, Justin Teller, Paul Cavallaro, Qi Huang, Justin Meza, and Kaushik Veeraraghavan. 2015. Gorilla: A fast, scalable, in-memory time series database. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1816–1827.
- [29] Paruj Ratanaworabhan, Jian Ke, and Martin Burtscher. 2006. Fast lossless compression of scientific floating-point data. In *Data Compression Conference (DCC'06)*. IEEE, 133–142.
- [30] Peter J Rousseeuw. 1987. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics* 20, 1 (1987), 53–65.
- [31] RSA. 2023. Spitzer Documentation & Tools. <https://irsa.ipac.caltech.edu/data/SPITZER/FLS/images/irac/>. Accessed May 1, 2025.
- [32] Michael Schindler. 1997. A fast block-sorting algorithm for lossless data compression. In *Proc. Data Compression Conf*, Vol. 469. Citeseer.
- [33] Philip Schmidt, Attila Reiss, Robert Duerichen, Claus Marberger, and Kristof Van Laerhoven. 2018. Introducing wesad, a multimodal dataset for wearable stress and affect detection. In *Proceedings of the 20th ACM international conference on multimodal interaction*. 400–408.
- [34] Julian Seward. 1996. bzip2 and libbzip2. *available at http://www.bzip.org* (1996), 8–18.
- [35] Seung Woo Son, Zhengzhang Chen, William Hendrix, Ankit Agrawal, Wei-keng Liao, and Alok Choudhary. 2014. Data compression for the exascale computing era-survey. *Supercomputing frontiers and innovations* 1, 2 (2014), 76–88.
- [36] Peter Thoman, Markus Wippler, Robert Hranitzky, and Thomas Fahringer. 2020. RTX-RSim: Accelerated Vulkan room response simulation for time-of-flight imaging. In *Proceedings of the International Workshop on OpenCL*. 1–11.
- [37] Robert Tibshirani, Guenther Walthner, and Trevor Hastie. 2001. Estimating the number of clusters in a data set via the gap statistic. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 63, 2 (2001), 411–423.
- [38] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [39] TPC. 2023. TPC-DS Version 2 and Version 3. <https://www.tpc.org/tpcds/default5.asp>. Accessed May 1, 2025.
- [40] Sebastian Wandelt, Xiaoqian Sun, and Ulf Leser. 2018. Column-wise compression of open relational data. *Information Sciences* 457 (2018), 48–61.
- [41] Xinyang Yu, Yanqing Peng, Feifei Li, Sheng Wang, Xiaowei Shen, Huijun Mai, and Yue Xie. 2020. Two-level data compression using machine learning in time series database. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1333–1344.